EXPERIMENT 1

Implement Iterative and Recursive Binary Search in C and Compare Their Performance

Name	HIMANSHU RAJ
Batch	B1 CCVT
SAP ID	590026147
Submitted To	Arjun Abhishek

B.Tech CSE – Cloud Computing & Virtualization (CCVT)

1. Aim

To implement iterative and recursive binary search algorithms in C and compare their performance using execution time and time-complexity analysis.

2. Objectives

• Implement binary search using an iterative approach.

• Implement binary search using recursion.

• Test both implementations on a sorted array.

• Measure and compare their execution times.

• Compare their time and auxiliary-space complexities.

3. Theory

Binary search is an efficient searching technique that works on a sorted array. It compares the search key with the middle element and eliminates half of the remaining search interval after each comparison. The iterative implementation repeats the process using a loop, while the recursive implementation calls the same function on the appropriate half of the array.

4. Algorithm

A. Iterative Binary Search

1. Set low = 0 and high = n − 1.

2. While low ≤ high, calculate mid = low + (high − low) / 2.

3. If arr[mid] equals the key, return mid.

4. If arr[mid] is less than the key, set low = mid + 1; otherwise, set high = mid − 1.

5. If the loop ends without a match, return −1.

B. Recursive Binary Search

1. If low > high, return −1 because the key is not present.

2. Calculate mid = low + (high − low) / 2.

3. If arr[mid] equals the key, return mid.

4. If the key is greater than arr[mid], recursively search from mid + 1 to high.

5. Otherwise, recursively search from low to mid − 1.

5. Sample Input and Output

Sample input: number of elements = 10; search key = 7. Both implementations find the element at zero-based index 6. The measured execution times may vary by computer, compiler, and system load.

6. Complexity Analysis

Parameter	Iterative	Recursive
Best-case time	O(1)	O(1)
Average-case time	O(log n)	O(log n)
Worst-case time	O(log n)	O(log n)
Auxiliary space	O(1)	O(log n)

7. Performance Comparison

Both methods have O(log n) time complexity. The iterative method generally uses less auxiliary memory because it does not create recursive function calls. The recursive method uses O(log n) call-stack space. For small inputs, the timer may display 0.00000000 seconds because of timer resolution; this does not mean the algorithms take literally no time.

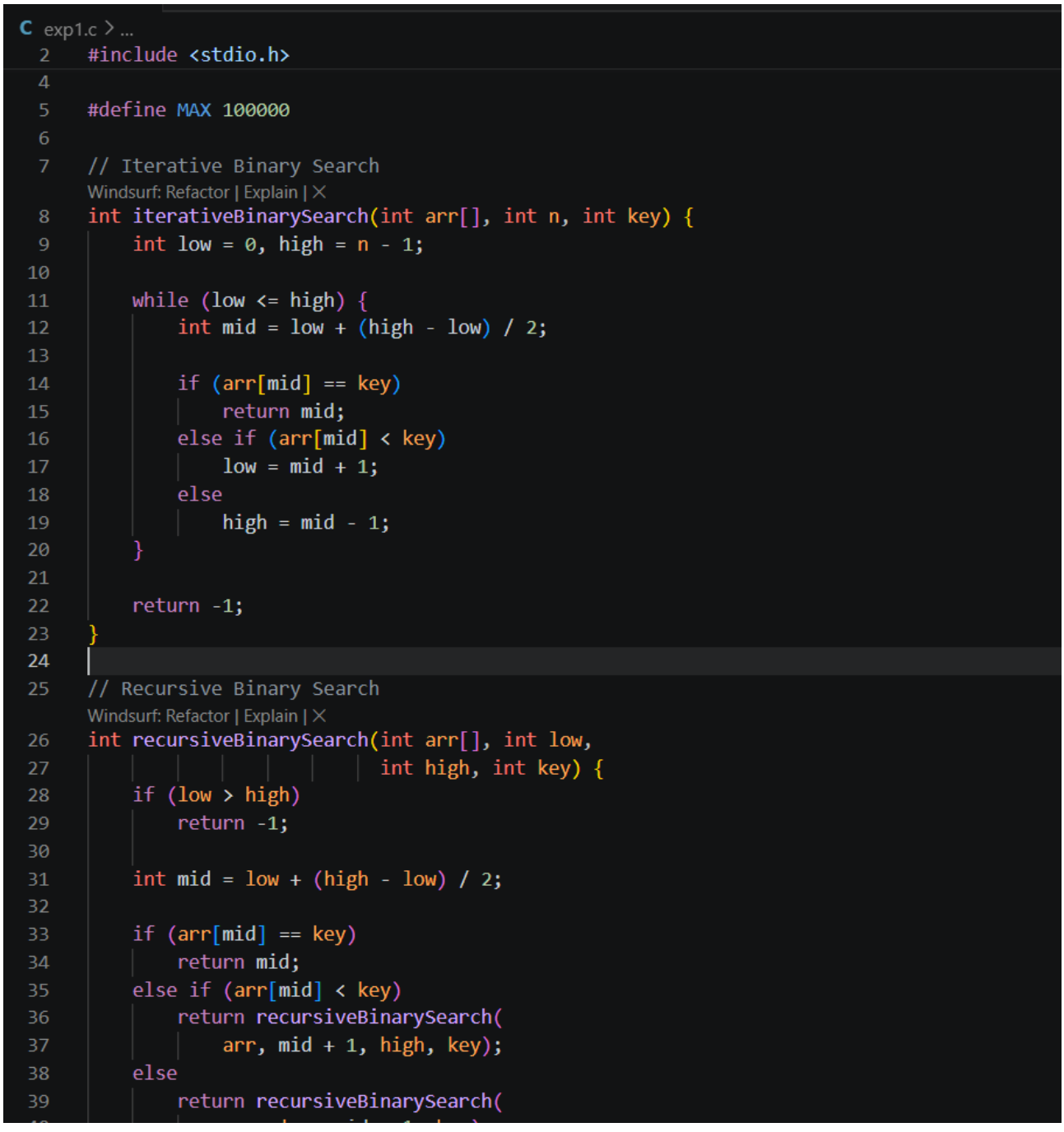
8. Result

The iterative and recursive binary search algorithms were implemented and executed successfully in C. For the test input of 10 sorted elements with key 7, both methods found the element at index 6. Both have O(log n) time complexity, while their auxiliary-space requirements differ.

9. Screenshots / Evidence

The following screenshots document the source code and successful execution in Visual Studio Code.

Screenshot 1



Screenshot 1: Iterative binary search function and beginning of recursive function in VS Code.

Screenshot 2

```c
// Recursive Binary Search
Windsurf: Refactor | Explain |
int recursiveBinarySearch(int arr[], int low,
                          int high, int key) {
    if (low > high)
        return -1;

    int mid = low + (high - low) / 2;

    if (arr[mid] == key)
        return mid;
    else if (arr[mid] < key)
        return recursiveBinarySearch(
            arr, mid + 1, high, key);
    else
        return recursiveBinarySearch(
            arr, low, mid - 1, key);
}

Windsurf: Refactor | Explain | Generate Function Comment |
int main() {
    static int arr[MAX];
    int n, key, result;
    clock_t start, end;
    double iterativeTime, recursiveTime;

    printf("Enter number of elements (1-%d): ", MAX);
    scanf("%d", &n);

    if (n < 1 || n > MAX) {
        printf("Invalid array size!\n");
        return 1;
    }

    // Create a sorted array
    for (int i = 0; i < n; i++)
        arr[i] = i + 1;
```

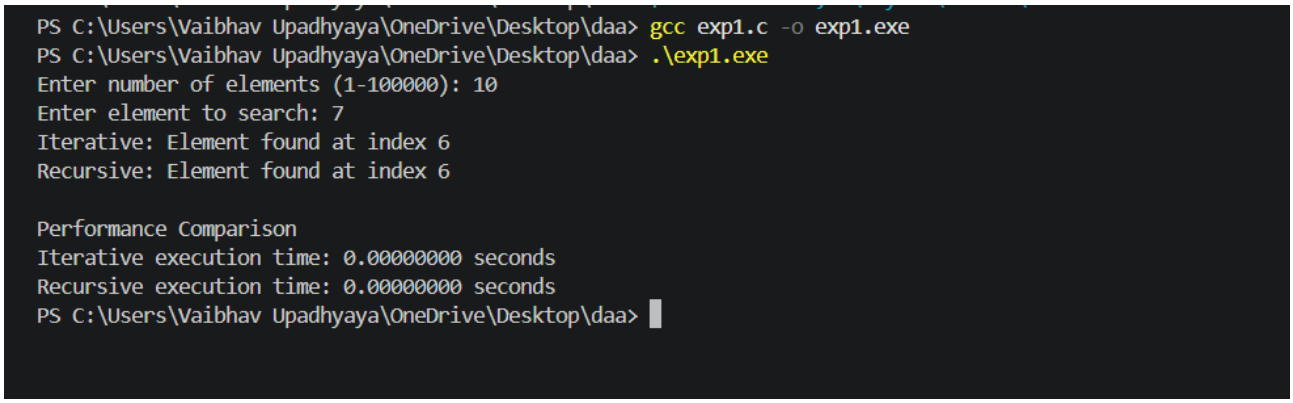
Screenshot 2: Recursive binary search function and beginning of main() in VS Code.

Screenshot 3

```
61      printf("Enter element to search: ");
62      scanf("%d", &key);
63
64      // Measure iterative search time
65      start = clock();
66      result = iterativeBinarySearch(arr, n, key);
67      end = clock();
68      iterativeTime = (double)(end - start) / CLOCKS_PER_SEC;
69
70      if (result != -1)
71          printf("Iterative: Element found at index %d\n", result);
72      else
73          printf("Iterative: Element not found\n");
74
75      // Measure recursive search time
76      start = clock();
77      result = recursiveBinarySearch(arr, 0, n - 1, key);
78      end = clock();
79      recursiveTime = (double)(end - start) / CLOCKS_PER_SEC;
80
81      if (result != -1)
82          printf("Recursive: Element found at index %d\n", result);
83      else
84          printf("Recursive: Element not found\n");
85
86      printf("\nPerformance Comparison\n");
87      printf("Iterative execution time: %.8f seconds\n",
88              iterativeTime);
89      printf("Recursive execution time: %.8f seconds\n",
90              recursiveTime);
91
92      return 0;
93  }
94
```

Screenshot 3: Remaining main() function and execution-time measurements.

Screenshot 4



Screenshot 4: Successful compilation and sample output for both search methods.